

Skill wise expert bases data generation

본 매뉴얼은 Lekiwi 환경 구축 및 원격조종 물체 스폰 매뉴얼 세팅이 완료되어있는 상태를 가정한다. 또한 전체 코드는 <https://app.notion.com/p/Skill-wise-expert-bases-data-generation-383072c6fe2280689f7edab449094a6d#39d56d61dfdb433890d90b7de3b57cbf>

페이지를 참고한다.

0. 실행환경

- Ubuntu 22.04.5 LTS
- Conda (Miniconda)
- Python 3.11 (env_isaac lab · Isaac 작업) / 3.12 (lerobot050 · 변환)
- Isaac Sim 5.0.0 / Isaac Lab 0.44.9
- PyTorch 2.7.0 + CUDA 12.8
- LeRobot 0.5.0 (HDF5 → LeRobot v3 변환)
- ROS2 Humble (텔레옴)

1. 운영 아키텍처

[Local PC 1]
So-arm 101 Leader arm 연결 후 조작 데이터 TCP 전송

[Local PC 2]
S0-arm 101 Leader arm 조작 데이터 수신 및 Isaac sim Lekiwi 로봇에 반영
Isaac sim 및 Isaac lab 환경 구축

[A100 40GB Server]
수집 데이터 기반 BC 진행

2. 파이프라인 개요

모방학습 기반의 정책을 학습시키기 위해서는 하나의 task 에도 최소 100 개 이상의 데이터가 필요하다. 하지만 이러한 데이터를 사람이 수동으로 수집하기에는 많은 시간과 노력이 필요하다. 이러한 문제를 해결하기 위해 task 당 10~20 개의 소량의 수동 조작 데이터를 수집해 먼저 모방학습으로 큰 틀에서의 행동을 정책이 수행하도록 만들고 이 모방학습 정책을 초기정책으로 사용하여 RL 로 초기정책에 보정값을 추가해 task 를 비교적 높은 성공률로 수행할 수 있는 skill wise expert 를 만들어 구축해둔 환경에서 정책을 구동해 모방학습을 위한 데이터를 수집한다.

Skill 은 Navigate, Approach&Lift, Carry, Approach&Place 4 가지로 구성하며 다음과 같은 순서로 진행한다

```
[1] 텔레옴    record_teleop.py          사람 시연 → HDF5
[2] BC 학습    train_diffusion_bc.py      Diffusion Policy (frozen base)
[3] RL 학습    train_resip.py             Residual PPO → RL Expert
[4] 평가      eval_navigate / eval_resip / eval_carry / eval_s3
[5] 데이터 수집 vllm/record_teleop_scene.py RL Expert 자동 rollout (ProcTHOR scene)
                                     → convert_hdf5_to_lerobot_v3.py → LeRobot v3 → Pi0.5
```

파인튜닝

스킬별 체크포인트 (BC / ResiP / obs)

Navigate - 26D

BC : dp_bc_nav_skill2_v4/dp_bc_epoch300.pt

RL : resip_nav_tucked_v4/resip_nav_best.pt

Approach&Lift - 30D

BC : dp_bc_small/dp_bc_epoch150.pt

RL : backup/approachandlift/resip64%.pt

Carry - 39D

BC : dp_bc_carry_v4/dp_bc_epoch300.pt

RL : resip_carry_v6/resip_carry_iter240.pt

Approach&Place - 55D

BC : dp_bc_skill3_55d_fixed_1e-4/dp_bc_epoch500.pt

RL : resip_s3_v25/resip_iter80.pt

3. 실행환경 구축

```
~/IsaacLab/                                # Isaac Lab 프레임워크
(isaac-sim/IsaacLab, editable)
└─ scripts/lekiwi_nav_env/                  # 이 프로젝트
(github.com/yubinws/lekiwi-nav-env @ stage1_server)
    └─ record_teleop.py                     # 텔레옴 수집
    └─ train_diffusion_bc.py                # BC (Diffusion Policy,
frozen base)
    └─ train_resip.py                       # RL (Residual PP0 → RL
Expert)
    └─ eval_navigate.py / eval_resip.py / eval_carry.py / eval_s3.py  #
스킬별 평가 (BC / BC+ResiP)
    └─ convert_hdf5_to_lerobot_v3.py        # HDF5 → LeRobot v3 변환
    └─ diffusion_policy.py / models.py      # 정책 네트워크
    └─ lekiwi_skill2_env.py / lekiwi_skill3_env.py / ...  # 스킬 환경
    └─ calibration/                         # tucked_pose.json,
arm_limits_measured.json (git tracked)
    └─ checkpoints/                        # BC + ResiP 체크포인트
(~39G, gitignore)
    └─ demos/ demos_skill2/ demos_skill3/   # 텔레옴 HDF5 (~220G,
gitignore)
    └─ vllm/                               # record_teleop_scene.py
등
    └─ setup.sh                             # 환경 설치 — 이 파일 하나로 전부
(bash setup.sh [server])
    └─ export/                              # 데스크탑 lockfile
(env_isaaclab.lock.txt, env_lerobot050.lock.txt)
    └─ secondserver/                       # 서버 lockfile
(env_rl_train.lock.txt)
~/Downloads/lekiwi_robot.usd                # 로봇 USD 래퍼 (7.3M) →
~/lekiwi/urdf/... 참조
~/lekiwi/                                  # 로봇 메시 트리 (4.7G)
~/isaac-objects/mujoco_scanned_objects/    # 물체 USD (3.8G)
~/molmospaces/                             # ProcTHOR scene 자산
(molmo_spaces_isaac, scene 1302 등)
~/miniconda3/envs/
└─ env_isaaclab                            # Isaac Sim 5.0 +
BC/RL/eval/record_scene (Python 3.11)
└─ lerobot050                              # LeRobot 0.5.0 변환
(Python 3.12)
```

명령어

실행환경 구축 — 아무것도 없는 PC에서 setup.sh 하나면 현재와 동일한 데이터 생성 환경이
순서대로 구축된다.

데스크탑: Miniconda → Isaac Lab → env_isaaclab(Isaac Sim 5.0) →
molmospaces(ProcTHOR) → lerobot050 → ROS2

```
명령어 : bash setup.sh
# BC/RL 학습을 A100 서버에 동일하게 구축시
명령어 : bash setup.sh server      # rl_train (Isaac Sim headless + skrl)
# 각 env 는 lockfile(pip freeze)로 정확 버전 고정 (2026-06-18 검증 시점과 동일):
#   export/env_isaacclab.lock.txt · export/env_lerobot050.lock.txt ·
secondserver/env_rl_train.lock.txt
# 기존 데스크탑 env 를 그대로 복제하려면: export/setup_env.sh 의 MODE=mirror 사용
```

setup.sh

[setup.sh — 노션 원문 참조]

export/env_isaacclab.lock.txt

[env_isaacclab.lock.txt — 노션 원문 참조]

export/env_lerobot050.lock.txt

[env_lerobot050.lock.txt — 노션 원문 참조]

secondserver/env_rl_train.lock.txt

[env_rl_train.lock.txt — 노션 원문 참조]

4. 환경·기반 설정

스킬을 학습하기 전 시뮬레이터에 로봇이 어떻게 정의되어있는지를 코드단에 먼저 정의해야 하고 또 각 스킬별로 obs 에 따라 어떤 값들을 필요로 하고 그 값들을 받아오기 위해 환경을 세팅하는 코드가 먼저 정의되어야 한다. 로봇 정의는 lekiwi_robot_cfg.py, 스킬별 환경은 lekiwi_skill2_env.py, lekiwi_skill2_eval.py, lekiwi_skill3_env.py 이 있다.

4.1 로봇 기반 설정 (lekiwi_robot_cfg.py)

LeKiwi 로봇을 Isaac Lab ArticulationCfg(LEKIWI_CFG)로 정의한다. 로봇은 이 파일에서 먼저 DEFINITION 되고(LEKIWI_CFG = ArticulationCfg), 각 스킬 env 가 LEKIWI_CFG.replace(prim_path=~)로 가져다 scene 에 배치한다.

로봇은 SO-100 6-DOF 팔(관절 5 + 그리퍼 1)과 3-wheel Kiwi 옴니 드라이브로 구성되고 제어 대상은 총 9 관절이다.

- **USD**: `usd_path` 는 환경변수 `LEKIWI_USD_PATH` 로 받고 없으면 로컬 기본 경로를 쓴다. 머신마다 USD 위치가 달라도 환경변수로 흡수한다.

- **초기 상태**: base 는 지면 5cm 위에서 드롭(`pos=(0,0,0.05)`, 안정화 후 약 3cm), 팔·그리퍼·바퀴 관절은 전부 0(rest)에서 시작한다.

- **actuator 3 그룹**(`ImplicitActuatorCfg`): 세 그룹 모두 `stiffness=None`, `damping=None` 인데, 이는 Isaac Lab 이 값을 덮어쓰지 않고 USD 에 박힌 drive 값을 그대로 쓴다는 뜻이다.

- **arm (force drive)**: 토크로 목표 관절각을 잡아주는 방식이다. 작은 `stiffness·damping`(USD 값 약 0.05~1.8 / 0.001~0.1)으로 팔 자세를 유지하고 ``set_joint_position_target()`` 으로 제어한다.

- **wheels (acceleration drive)**: `stiffness=0`(위치 제어 없음) + 큰 `damping(174.53)`으로 바퀴 각속도를 제어한다. ``set_joint_velocity_target()``으로 명령하면 Kiwi IK 가 base 속도를 실현한다.

- **rollers (passive)**: 옴니휠 측면 롤러로, DriveAPI 없이 자유 슬립만 담당한다.

- **Kiwi IK 상수**: 바퀴 반지름 0.049m, base 반지름 0.1085m, 바퀴 각도 -30°, -150°, 90°. base 속도 (`vx, vy, wz`)를 3×3 행렬로 세 바퀴 각속도로 변환한다.

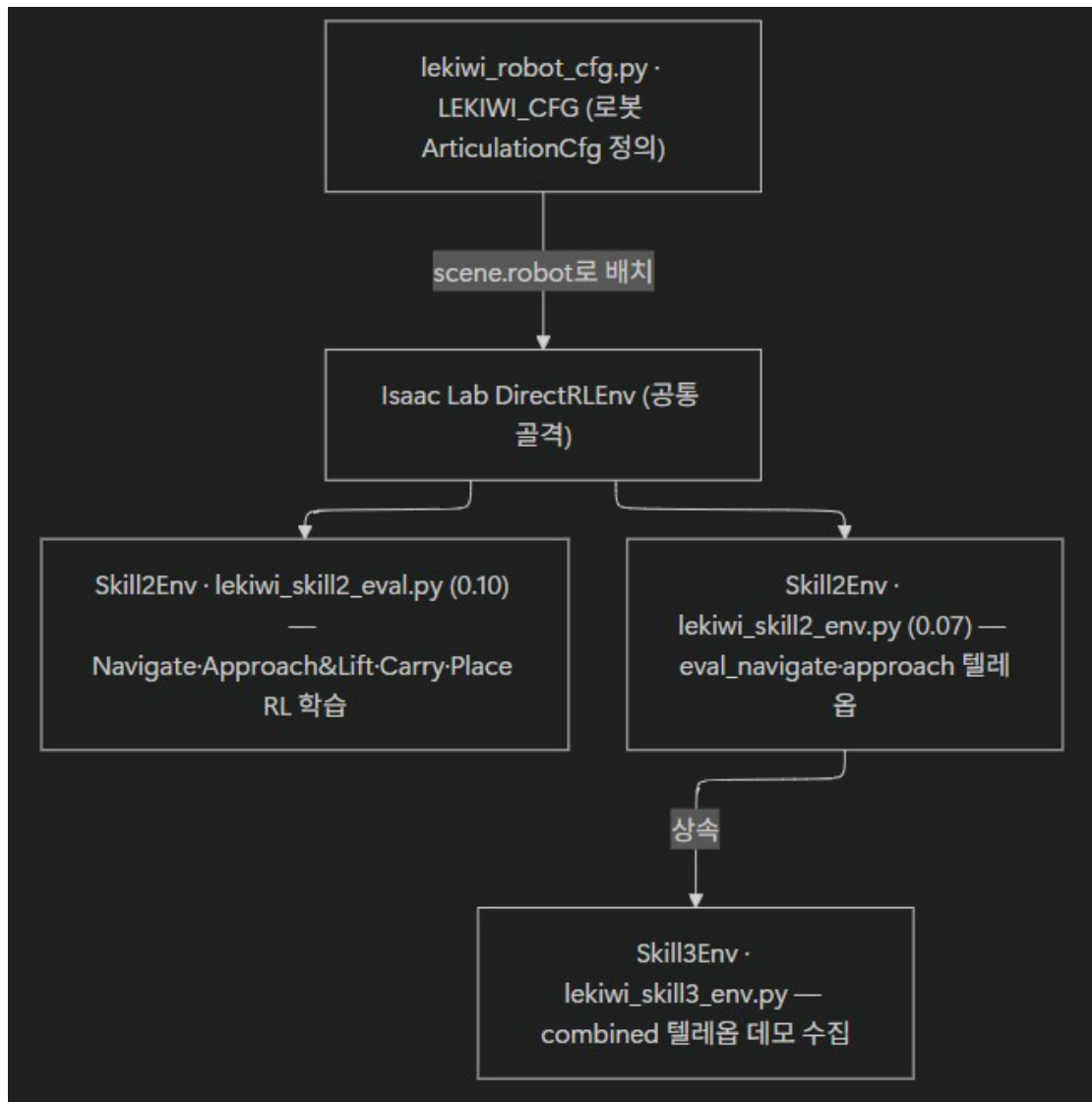
- **팔 soft limit**: 실측 캘리브레이션값(``calibration/arm_limits_measured.json``)을 baked limit 으로 갖고 env 생성 시 PhysX 에 적용해 자기 충돌을 막는다.

lekiwi_robot_cfg.py

[lekiwi_robot_cfg.py — 노션 원문 참조]

4.2 환경 공통 구조 (Isaac Lab DirectRLEnv)

모든 스킬 env 는 Isaac Lab ``DirectRLEnv``를 상속하고, GPU 에서 수천 개(`num_envs`)를 동시에 시뮬레이션한다.



실제 사용되는 env 파일은 3 개이며 하나로 합쳐져 있지 않다.

- lekiwi_skill2_eval.py : **RL 학습(train_resip)의 모든 스킬이 이 env 를 쓴다** —

approach·carry·navigate ResiP 전부 여기서 인스턴스화한다. 데이터 수집,
eval_resip/eval_carry/eval_s3 도 이 파일.

- lekiwi_skill2_env.py : eval_navigate, record_teleop 의 approach_and_grasp 브랜
치 전용.

- lekiwi_skill3_env.py : record_teleop `--skill combined` 텔레옴 데모 수집 전용
(carry/place 학습 env 가 아니다).

즉 **RL 학습 = 항상 Skill2Env(lekiwi_skill2_eval, 0.10)**이고, 0.07 env·skill3 env 는
텔레옴 수집/평가 용도로만 쓴다.

환경이 정의하는 것은 다음과 같다.

- **sim**: physics dt 0.02(50Hz), decimation 2 → 정책 제어 주기는 0.04s(25Hz)다. device 는 Skill2/3=cpu 가 기본이라 RL 학습 때는 cuda 로 바꿔준다.

- **scene**(`\_setup\_scene`): 로봇 + 물체 + 지면 + (필요시) 카메라·contact sensor 를 만들고 N 개 env 로 복제한다.

- **action 9D**(lekiwi_v6 순서): `[0:5]` 팔 관절, `[5]` 그리퍼, `[6:8]` base 선속도 (vx,vy), `[8]` base 각속도(wz). action 은 body frame 으로 해석하고 `\_apply\_action` 에서 body→IK 변환(ik_vx=body_vy, ik_vy=-body_vx, ik_wz=body_wz) 후 바퀴 각속도로 바꾼다. 팔은 action 을 관절 한계로 매핑해 position target 으로 준다. action delay 버퍼로 1 스텝 지연(실로봇 SSH/ZMQ 지연 모사)도 넣는다.

- **obs**(`\_get\_observations`): actor 용과 critic 용 obs 를 따로 낸다(AAC 구조 — critic 은 학습 때만 쓰는 추가 정보를 더 본다). 학습 중에는 obs 에 noise 를 더한다.

- **reward / done / reset**: 스킬별 보상을 합산하고, 종료 판정(범위 이탈·넘어짐·스킬 성공)을 하며, 종료된 env 만 재초기화한다.

4.3 Skill2Env — Navigate

navigate 학습·검증 env 는 Skill2Env(lekiwi_skill2_eval)다

train_resip 의 main_navigate 가 `from lekiwi\_skill2\_eval import Skill2Env` 로 Skill2Env(cfg)를 인스턴스화한다(collect_nav_skill2·eval_navigate 도 동일). 방향 명령(direction_cmd)을 따라 base 만 움직이며, 팔은 init_arm 자세로 고정하고 RL 은 바퀴 보정만 한다.

obs

- Skill2Env env 자체는 actor 30D / critic 37D 지만, navigate ResiP 는 env 의 30D obs 를 직접 쓰지 않고 자기 26D obs 를 따로 구성한다.

- BC/residual 26D(실제 학습 obs) = arm(5) + gripper(1) + base 속도(선속도 xy 2 + 각속도 wz 1, 3D) + 방향 명령(3) + lidar(8, dummy ones) + init_arm(6 = arm5 + gripper1). 실제 lidar 는 쓰지 않는다(BC 체크포인트 dp_bc_nav_skill2_v4 obs_dim=26).

구간	항목	설명
[0:5]	arm 관절 위치 5D	팔 5관절
[5:6]	gripper 1D	그리퍼 위치
[6:9]	base 속도 3D	선속도 xy + 각속도 wz
[9:12]	direction_cmd 3D	6방향 명령
[12:20]	lidar 8D	dummy(1.0 고정, 실제 미사용)
[20:26]	init_arm_pose 6D	진입 시 팔자세(arm5+gripper1)

reward

env 가 아니라 train_resip.py main_navigate 가 매 step 직접 계산한다. tracking + tucked-pose 유지만 쓰고 장애물 보상은 없다(장애물은 배포 때 VLM 이 판단).

항목	가중치	정의
linear vel tracking	1.5	$\exp(-\text{lin_err}/0.25^2)$, 선속도 오차 (std 0.25 m/s)
angular vel tracking	1.5	$\exp(-\text{ang_err}/0.25^2)$, 각속도 오차 (std 0.25 rad/s)
tucked-pose hold	2.0 / 0.5	$2.0 \cdot \exp(-\text{arm_err}/0.3^2) + 0.5 \cdot \exp(-\text{gripper_err}/0.2^2)$, init_arm 자세 유지
action smoothness	-0.005 / -0.01	$-0.005 \cdot \Delta \text{base}^2$ (3채널) $-0.01 \cdot \Delta \text{arm}^2$ (6채널)
time penalty	-0.01	baseline

방향 개수는 단계마다 다르다.

BC 데모 수집(collect_nav_skill2)은 **6 방향**(전진/후진/좌·우 strafe + turn 좌/우)을 수집해 turn 까지 BC 가 학습하고, RL residual(train_resip main_navigate)은 **4 방향**(전진/후진/좌·우 strafe, turn 제외 — "turn 은 BC 로 충분")만 reset 마다 고정 샘플한다.

residual/critic

ResidualPolicy(diffusion_policy.py)를 쓰며, actor·critic 입력 = normalized obs 26D + base action 9D = 35D(RES_DIM). per_dim action scale = arm 0.05 / grip 0.05 / base 0.0(base 는 BC 그대로).

4.4 Skill2Env — Approach & Lift

Navigate 로 물체 근처까지 접근했다면 물체를 팔로 실제로 잡을 수 있을 정도의 거리까지 정밀하게 접근해 잡고 들어올린다. fixed joint 없이 순수 마찰로 grasp 한다.

obs

- env actor 30D = arm(5) + grip(1) + base 속도(3) + 선속도(3) + 각속도(3) + 팔·그리퍼 관절속도(6) + 물체 상대위치(3) + contact L/R(2) + bbox(3) + 카테고리(1). critic 37D = actor + bbox(3) + 질량(1) + 거리(1) + heading(1) + 접근속도(1).

- BC 30D = env actor obs 그대로.

데모 obs = 학습 obs : BC 데모는 통합 텔레옴(`record\_teleop.py --skill combined`)으로 수집한다. 텔레옴 물리 env 는 Skill3Env(lekiwi_skill3_env)지만 이 클래스는 Skill2Env를 상속하므로 obs 구조가 동일하다. 게다가 record_teleop 는 phase1(approach&lift) obs 를 30D(Skill2Env 와 같은 형식)로 따로 저장한다(phase1_obs, s2_obs_pre 30D). 즉 저장된 데모 obs = 학습 env(Skill2Env, 30D/37D) obs 로, 분포가 다르지 않다.

actor 30D:

구간	항목	설명
[0:5]	arm pos 5D	팔 관절 위치
[5:6]	grip pos 1D	그리퍼 위치
[6:9]	base body vel 3D	vx, vy, wz
[9:12]	base lin vel 3D	선속도
[12:15]	base ang vel 3D	각속도
[15:21]	arm+grip joint vel 6D	관절 속도
[21:24]	object rel pos 3D	물체 상대위치(body)
[24:26]	contact L/R 2D	jaw, wrist 접촉
[26:29]	object bbox 3D	물체 크기
[29:30]	object category 1D	물체 종류

critic 37D = actor 30D + 다음 7D:

구간	항목	설명
[30:33]	bbox 원본 3D	비정규화
[33:34]	object mass 1D	질량
[34:35]	object_dist 1D	물체 거리
[35:36]	heading 1D	물체 방위
[36:37]	vel_toward 1D	접근 속도

reward

항목	가중치	정의
time	-0.01	baseline
effort	-0.01	base ²
arm_move	-0.02	arm_vel ² (6D)
smoothness	-0.005	$\Delta \text{action}^2(9D)$
approach_progress	6.0	clamp(이전거리 - 거리, ± 0.2)
proximity_tanh	2.0	$1 - \tanh(\text{거리}/0.5)$
heading	0.2	$\cos(\text{물체 heading})$
vel_toward	0.5	물체로 향하는 속도
grasp 성공	+10	just_grasped
lift 성공	+5	task_success(들어올림)
gripper shaping	—	3-band: 멀면 $+0.1 \times \text{open}$, 0.35~0.70m $+0.3 \times \text{close}$, 가까우면 $+0.5 \times \text{close}$
height penalty	-0.1	clamp(물체 z - 0.10)

grasp 판정: 그리퍼 닫힘(<0.65) + contact force(>0.55N) + EE↔물체 거리(<0.10m, 학습 env=lekiwi_skill2_eval 기준)가 동시 성립(eval env 는 jaw+wrist 이중 contact 와 inside_bbox 도 추가). 0.07m 는 텔레옴·eval_navigate 전용 lekiwi_skill2_env 값이다.
lift 성공 = 물체가 0.05m 이상 상승.

4.5 Skill2Env — Carry

물체를 잡고있는 상태로 목표 물체 근처까지 운반하는 스킬이다.

carry 학습·검증 env 는 Skill2Env(lekiwi_skill2_eval)다 — train_resip 의 `--skill`

`carry`` 는 make_env 에서 Skill2Env 를 인스턴스화한다

(approach_and_grasp·combined_s2_s3·carry 모두 Skill2Env). carry 는 base 운반

이 핵심이라 approach 환경을 그대로 재사용하고 사전 설정해둔 성공 조건을 충족하면 자동으로 carry 학습이 진행되는 구조이다.

obs

- env actor 30D / critic 37D (Skill2Env) = arm(5) + grip(1) + base 속도(3) + ...

(Approach & Lift 와 동일 구조). critic 37D = actor 30D + bbox(3) + 질량(1) + object_dist/heading/vel_toward 등 추가 7D.

- **BC 39D = Skill2Env actor 30D + 방향 명령(3) + init_arm(6)**. carry 정책은 Skill2Env 30D obs 를 기반으로 학습·배포한다.

- **데모 수집**: 텔레옴이 아니라 `collect\_carry.py` (자동, S2 expert 가 들어올린 상태에서 direction-conditioned 로 수집)가 생성한다. 수집 env 도 Skill2Env(eval)로 학습 env 와 동일하며, 출력 obs 는 39D 다.

구간	항목	설명
[0:30]	Skill2Env actor 30D	4.4 actor와 동일
[30:33]	direction_cmd 3D	운반 방향
[33:39]	init_arm_pose 6D	lift 시점 팔자세(arm5+grip1)

Reward

carry 보상은 env 가 아니라 train_resip.py(`--skill carry`, main_carry)에서 계산한다. 명령된 방향(direction_cmd)으로의 body-velocity tracking 구조이며(navigate 와 동일), 아래 항목의 합이다. (lekiwi_skill3_env 의 _get_rewards — carry_progress 3.0 / place +20 등 — 는 carry_and_place 전용이라 carry 학습에 쓰이지 않는다.)

항목	가중치	정의
rew_lin	1.5	$1.5 \times \exp(-\text{lin_err}/0.075^2)$, $\text{target} = \text{direction_cmd} \times 0.15(\text{ramp})$
rew_ang	1.5	$1.5 \times \exp(-\text{ang_err}/0.10^2)$, $\text{target_wz} = \text{direction_cmd} \times 1.0(\text{ramp})$
rew_smooth	-0.005 / -0.01	$-0.005 \times \Delta \text{base}^2 - 0.01 \times \Delta \text{arm}^2$
rew_hold	0.05	objZ > 0.05면 0.05
rew_arm_hold	2.0 / 0.5	$2.0 \times \exp(-\text{arm_err}/0.3^2) + 0.5 \times \exp(-\text{grip_err}/0.2^2)$, init_arm_pose 유지
rew_time	-0.01	baseline
rew_drop	-5.0	objZ < 0.03(drop)면 -5.0 + S2로 강제 리셋

4.6 Approach & Place

dest 옆에 들고있던 물체를 정밀하게 내려놓는 스킬이다. carry 보다 정밀해야 해서 obs·reward·구조가 모두 다르다.

Env

학습·배포 env 는 모두 Skill2Env 다. (train_resip `--skill combined\_s2\_s3` → make_env 에서 Skill2Env, 30D actor / 37D critic; eval_s3.py 도 동일)

lekiwi_skill3_env(Skill3Env, 29D)는 carry_and_place(BC eval/replay) 전용이며 RL 학습에는 쓰이지 않는다.

데모 obs = 학습 obs : place BC 데모는 같은 통합 텔레옴(`record\_teleop.py --skill combined`)의 phase3 를 수집한다. 텔레옴 물리 env 는 Skill3Env(lekiwi_skill3_env)지만 Skill2Env 를 상속한다. record_teleop 는 phase3 obs 를 build_s3_bc_obs(phased53 → 55D)로 저장하므로, 저장된 데모 obs = place BC 학습 obs(55D)로 분포가 동일하다.

obs 55D

build_s3_bc_obs 가 런타임에 구성한다: Skill3 obs(29) + 진입 시 팔자세 init_pose(6) + phase_a flag(1) + 확장 16(source 상대위치 · source→dest · 수평거리 · 높이 · upright · holding · obj_up · source→gripper) + place_open flag(1) + release flag(1) + retract flag(1).

구간	항목	설명
[0:21]	base obs 21D	arm5+grip1+base_vel3+lin3+ang3+armvel6
[21:24]	dest_rel_body 3D	목적지 상대위치
[24:25]	contact_force 1D	접촉력
[25:28]	bbox 3D	물체 크기
[28:29]	category 1D	물체 종류
[29:35]	init_pose 6D	진입 시 팔자세
[35:36]	phase_a flag 1D	A/B phase
[36:39]	source_rel_body 3D	물체 상대위치
[39:42]	source_to_dest_body 3D	물체->목적지
[42:43]	source_to_dest_xy 1D	수평거리
[43:44]	source_height 1D	물체 높이
[44:45]	source_upright 1D	물체 기립도
[45:46]	source_holding 1D	파지 여부
[46:49]	source_up_body 3D	물체 up벡터(body)
[49:52]	source_to_gripper_body 3D	물체->그리퍼
[52:53]	place_open flag 1D	놓기 트리거
[53:54]	release flag 1D	릴리즈
[54:55]	retract flag 1D	후퇴

4-phase ABCD 상태머신 — env 가 아니라 eval/expert(`eval\_s3.py` / PlaceExpert)에 있다. 단계마다 residual 보정 한도(scale)가 다르다.

phase	팔	그리퍼	바퀴	의미
A 접근	0	0	0.40	dest로 이동
B 하강	0.05	0 (조건부 0.50)	0.20	팔 내려 정렬
C release	0.05	0.50	0.05	그리퍼 열어 놓기
D retract	0.10	0.30	0.05	팔 빼고 base 정지

Reward

env 의 `_get_rewards(+20/-10)`가 아니라 `train_resip` combined 커리큘럼으로 학습한다:
 place 성공 +200 + 300×`arm1_quality`(팔 접힘 품질), `drop·timeout` -5, `rest-pose` shaping, `time` -0.01. place 성공 = `source↔dest` 수평거리 <0.18m + 비접촉 + 적정 높이 + `upright` + 팔 접힘 + 그리퍼 열림이 3 스텝 연속. (env 의 place 거리 0.20 과 학습 판정 0.18 은 별개 값이다.)

항목	가중치	정의
place 진전	+20 x <code>place_delta</code>	<code>place_delta</code> = <code>clamp</code> (이전거리 - 배치 오차, -0.05, +0.05)
place 성공	+200 + 300 x <code>arm1_quality</code>	<code>source↔dest XY</code> < 0.18(<code>S3_PLACE_RADIUS</code>), <code>arm1_quality</code> = <code>clamp</code> ((<code>arm1_max</code> - 2.5)/0.5, 0, 1)
M2 tiers	+10 / +30 / +60 / +150	거리 단계별 one-time 보너스
rest-pose shaping	—	팔 접힘(<code>arm1_quality</code>) 기반 자세 유도
drop	-5	물체 낙하
timeout	-5	시간 초과
time	-0.01	baseline

5. BC, RL 구조

모든 스킬 expert 는 BC + **ResiP(Residual Policy)** 구조다

Diffusion Policy BC 가 텔레옴을 모방해 목표 행동을 어느정도 수행하는 base 궤적을 내고 그 위에 **Residual PPO** 가 작은 보정만 학습해 행동을 다듬고 성공률을 끌어올린다. 순수 BC 는 데모의 평균적이라 정밀도가 부족하고 순수 RL 은 9D action 에서 reward 탐색 및 수렴이 어려워 둘을 결합한다.

5.1 BC : Diffusion Policy (U-Net 1D)

BC(Behavior Cloning)는 사람의 텔레옴 시연으로부터 "지금 관측(obs)을 보고 다음에 어떤 action 을 낼지"를 배우는 모방학습이고, 여기서는 이 정책을 Diffusion Policy 로 학습한다.

왜 단순 회귀가 아니라 diffusion 인가

같은 상황에서도 정답 action 이 하나가 아니기 때문이다. 약병을 피해 갈 때 왼쪽으로 돌아도 되고 오른쪽으로 돌아도 되는데, 관측 하나에 action 하나를 바로 맞히도록 MSE 회귀로 학습하면 네트워크는 왼쪽 정답과 오른쪽 정답의 평균을 출력하고, 그 평균은 가운데로 직진하는 어느 쪽도 아닌 행동이라 결국 물체에 부딪힌다. Diffusion Policy 는 action 을 하나로 콕 집어 예측하는 대신 정답들이 퍼져 있는 분포 전체를 학습하기 때문에 왼쪽 길과 오른쪽 길 같은 여러 모드를 평균 내지 않고 그대로 표현할 수 있다.

학습 방식 (DDPM)

이미지 생성 AI 처럼 노이즈에서 점점 깨끗한 action 으로 복원하도록 학습하며 그 표준 절차가 DDPM(Denoising Diffusion Probabilistic Models)이다. 먼저 진짜 action 에 노이즈를 100 단계에 걸쳐 조금씩 섞는데, 임의의 단계 k 에서의 노이즈 섞인 action 은 한 번에 이렇게 만들 수 있다.

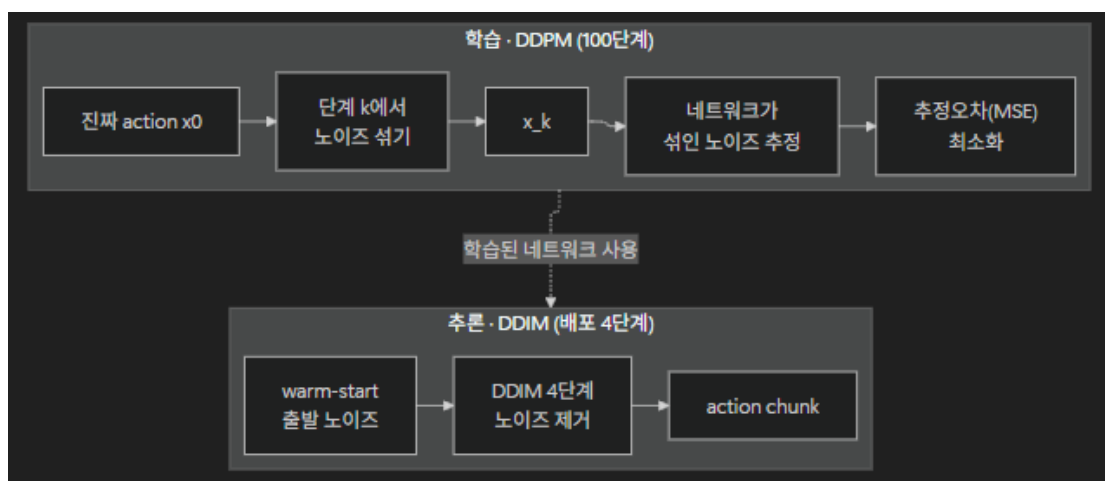
$$x_k = \sqrt{\bar{\alpha}_k} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_k)} \cdot \epsilon \quad (\epsilon \sim N(0, I))$$

x_0 는 진짜 action(정규화된 값), ϵ 는 무작위로 뽑은 노이즈이고 $\bar{\alpha}_k$ 는 noise schedule 이 정하는 0~1 계수다. k 가 커질수록 $\bar{\alpha}_k$ 가 0 에 가까워져 원본은 사라지고 노이즈만 남으며 (k=0 이면 거의 원본, k=100 이면 거의 순수 노이즈), schedule 은 cosine 방식 (squaredcos_cap_v2)에 총 100 단계(num_diffusion_iters=100)다. 그 다음 무작위 단계 k 를 뽑아 위 식으로 x_k 를 만들고 네트워크 $\epsilon_\theta(x_k, k, obs)$ 가 그때 섞인 노이즈 ϵ 이

무엇이었는지 맞히도록 학습한다(noise/epsilon 예측). 손실은 예측한 노이즈와 실제 노이즈의 차이(MSE)이고, 이렇게 학습하면 네트워크가 노이즈를 지우는 법을 배운다.

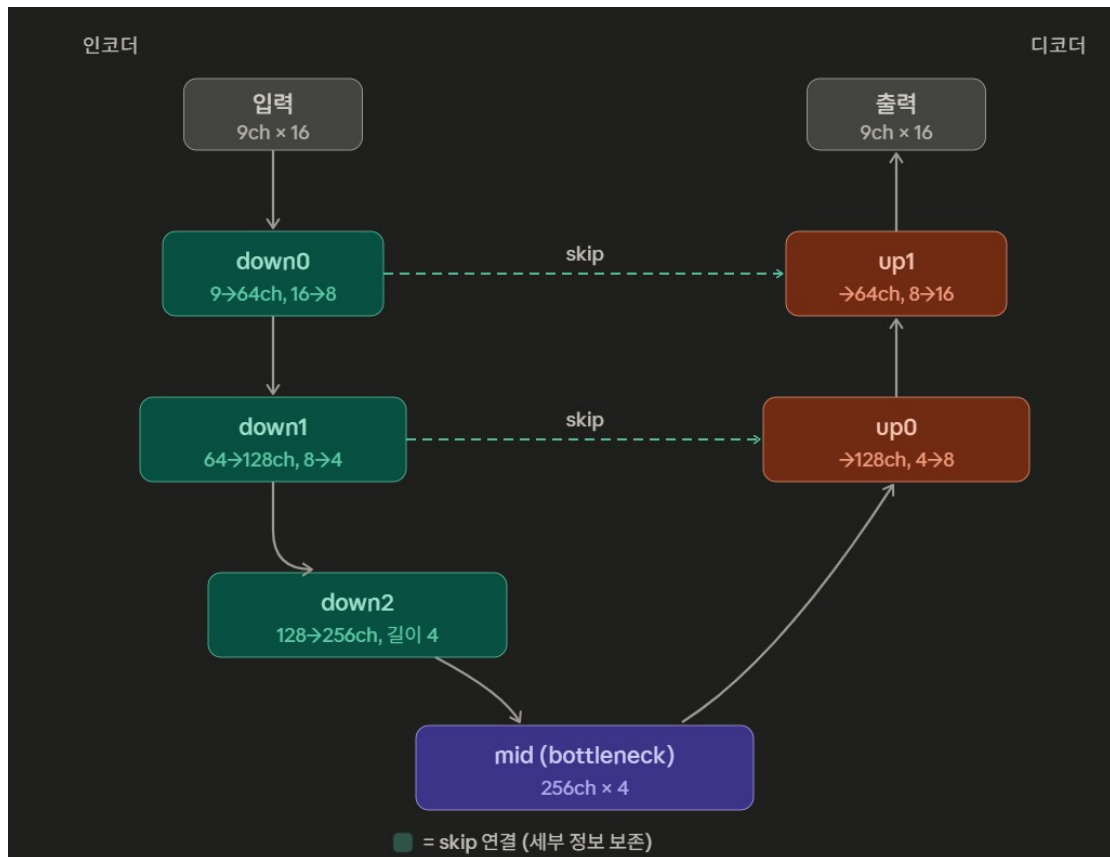
추론 방식 (DDIM)

학습은 DDPM 100 단계로 꼼꼼하게 하지만 추론까지 100 단계를 다 거치면 매 제어마다 네트워크를 100 번 돌려야 해서 너무 느려 25Hz 실시간 제어에는 못 쓴다. 그래서 추론에는 DDIM(Denoising Diffusion Implicit Models)을 쓰는데, 학습한 네트워크는 그대로 두고 복원 단계를 다 거치지 않고 몇 단계만 건너뛰며(배포 시 4 단계) 결정론적으로 복원하기 때문에 품질을 거의 유지하면서 추론이 수십 배 빨라진다. 여기에 매번 순수 노이즈에서 새로 시작하지 않고 직전 추론 결과를 중간 단계 노이즈로 되돌려 출발점으로 재사용하는 warm-start 를 더해 4 단계만으로도 안정적으로 수렴하게 한다. 결국 DDPM 은 어떻게 학습하느냐 (노이즈 예측)이고 DDIM 은 학습된 네트워크를 어떻게 빨리 쓰느냐(적은 단계 복원)로, 같은 네트워크에 샘플링 방식만 다른 것이다.



왜 U-Net 이고 왜 1D 인가

노이즈를 잘 지우려면 궤적의 큰 흐름과 세부 값을 동시에 봐야 하고 그걸 가장 잘하는 구조가 U-Net 이라 이미지 diffusion 도 U-Net 을 쓴다. U-Net 은 U 자 모양의 인코더-디코더로, 인코더(down)에서 입력의 시간 길이를 절반씩 줄여 큰 흐름을 압축하면서 채널은 늘리고(down_dims [64,128,256]은 단계마다 채널을 64→128→256 으로 키운다는 뜻), 디코더(up)에서 다시 길이를 키워 세부 값을 복원하며, 인코더의 같은 길이 단계 정보를 디코더로 건너뛰어 잇는 skip 연결이 줄였다 키우는 과정에서 잃기 쉬운 세부를 보존한다.



각 블록 내부는 Conv1d → GroupNorm → Mish(kernel 5)로 이루어진다.

Conv1d(kernel 5)는 연속한 5 개 시점을 한 묶음으로 훑어 행동이 시간에 따라 어떻게 변하는지를 잡고, GroupNorm 은 채널을 그룹으로 묶어 정규화해 배치가 작아도 안정적이라 BatchNorm 대신 diffusion 에서 표준으로 쓰며, Mish 는 ReLU 를 매끄럽게 만든 활성화 함수로 gradient 가 부드러워 학습이 안정적이다.

1D 인 이유는 우리가 복원하는 게 이미지(2D)가 아니라 시간에 따른 행동의 흐름이기 때문이다. 한 번에 다루는 데이터는 (시간 16 × action 9 채널) 모양의 텐서이고(Conv1d 입력은 채널-우선이라 9 채널 × 길이 16), 이미지처럼 가로·세로 2 차원 공간 구조가 있는 게 아니라 시간이라는 1 개 축 위에 9 개 채널(팔 5·그리퍼 1·바퀴 3)이 얹힌 형태라 커널이 시간 축만 따라 미끄러지는 1D U-Net 을 쓴다.

obs 와 단계 정보 반영 (FiLM)

같은 노이즈라도 지금 상황(obs)과 지금 몇 단계째인지(k)에 맞는 action 으로 복원되어야 하므로 이 두 정보를 네트워크 각 층에 주입하는데, 이때 FiLM(Feature-wise Linear

Modulation) 방식을 쓴다. 단계 번호 k 는 사인파 임베딩(SinusoidalPosEmb)으로 벡터화하고 여기에 obs 를 합쳐 조건 벡터를 만드는데, 우리 obs 는 관절 각도·속도 같은 저차원 상태 벡터라 이미지처럼 CNN 으로 인코딩할 필요 없이 그대로 조건 벡터에 합쳐지고 현재 시점 관측 하나만 본다(obs_horizon=1). 이 조건 벡터를 작은 MLP 에 통과시켜 층마다 $\text{scale}(\gamma)$ 과 $\text{shift}(\beta)$ 를 뽑고 각 블록 출력을 출력 $\times \gamma + \beta$ 로 변형하기 때문에, 같은 노이즈라도 obs 와 단계에 따라 다른 action 으로 복원된다.

묶음 예측(chunk)과 receding horizon

매 스텝 action 하나씩 내지 않고 한 번에 pred_horizon=16 스텝을 예측한 뒤 그중 action_horizon=8 스텝만 실행하고 다시 관측해 재예측한다(receding horizon). 묶음으로 예측하면 동작이 시간적으로 일관되고 부드러워 한 스텝씩 낼 때 쌓이는 오차가 줄고, 8 스텝마다 다시 예측하기 때문에 한 번의 예측에 너무 의존하다 빗나가는(drift) 것도 막는다.

정규화와 학습 디테일

obs 와 action 은 학습 전에 LinearNormalizer 로 [-1,1] 범위로 min-max 정규화하고 네트워크 출력은 다시 원래 단위로 역정규화한다. diffusion 은 데이터가 표준 범위에 있을 때 안정적이라 이 정규화가 필수이며, § 11.3 에서 이 정규화 통계를 보정하는 이유와 직접 연결된다. 학습은 train_diffusion_bc.py 로 epochs 500 · batch 256 · lr 1e-4(cosine annealing) · EMA 0.995 로 돌리는데, EMA(지수이동평균)는 매 스텝 가중치를 $\text{EMA} = 0.995 \cdot \text{EMA} + 0.005 \cdot \text{현재}$ 로 천천히 평균낸 별도 사본을 유지하는 것으로 학습 중의 흔들림을 걸러 더 안정적인 모델을 만들고 배포 체크포인트로는 이 EMA 가중치를 쓴다.

핵심 수치 (참고용 — 실제 설정값):

```
네트워크 : ConditionalUnet1D (블록 Conv1d→GroupNorm→Mish, kernel5) ·
down_dims [64,128,256]
diffusion: DDPM 학습(noise 예측, 100 step) / DDIM 추론(배포 4 step, warm-
start)
chunk      : 16 예측 → 8 실행 → 재예측 / 정규화 LinearNormalizer [-1,1]
학습       : train_diffusion_bc.py · epochs 500 · batch 256 · lr 1e-4 · EMA
0.995
ckpt cfg   : {obs_dim, act_dim, pred_horizon 16, action_horizon 8,
num_diffusion_iters 100, down_dims}
```

5.2 RL — Residual PPO

우리가 사용하는 로봇은 Lekiwi 로봇으로 arm 6D + wheel 3D 총 9D 이다. 서로 행동 공간이 다른 9D action 을 scratch RL 로 진행하게 되면 reward 가 너무 sparse 하고 또 9D 라는 행동공간이 크기 때문에 매우 정교한 보상 설계가 필요하고 이를 완성하는데 매우 오랜 시간이 걸린다.

그래서 residual(보정)만 RL 로 배운다. BC 를 버리지 않고 freeze 시켜둔 뒤 이를 base 정책으로 두고 그 위에 작은 보정값만 RL 로 학습한다. 최종 action = BC 의 base action + 작은 보정으로 RL 은 행동 전체가 아니라 BC 가 살짝 모자란 부분만 배우면 되니 보상설계 및 학습이 쉽고 BC 에서 잘 되던 부분을 망가뜨리지 않는다.

보정값을 만드는 과정.

1. 고정된 BC 가 base action 을 낸다.
2. 작은 신경망(ResidualPolicy, 2 층 MLP)이 설정한 OBS + BC 의 base action 을 입력받아 보정 벡터를 출력한다.
3. 최종 action = base action + 보정 $\times$ per_dim_scale.
 - BC 는 모든 값을 [-1,1]로 정규화한 공간에서 동작하므로 보정도 같은 정규화 공간에서 더한다. 보정이 폭주해 base 를 망가뜨리지 않게 먼저 [-1,1]로 clamp 한 뒤 per_dim_scale 을 곱하고 마지막에 역정규화 해 실제 로봇에 사용할 수 있는 단위로 되돌린다.
 - 이때 per_dim_scale 은 차원마다 보정을 얼마나 허용할지의 한도다
 - 예시로 navigate 는 base 바뀌는 BC 출력을 그대로 쓰고(바뀌 0.0) 팔·그리퍼만 자세 drift 보정용으로 작게 0.05 씩 주며, approach&lift 는 base 는 0.35, arm 은 0.2, gripper 는 0.25 로 진행, approach&place 는 접근→하강→놓기→후퇴 4 단계로 나뉘어 각 단계마다 보정 한도를 다르게 준다. 이는 보정이 base 를 살짝 다듬는 수준을 넘지 않게 막는 장치다.
4. 이 보정 신경망은 PPO 로 학습된다. PPO(Proximal Policy Optimization)는 강화학습에서 정책을 한 번에 너무 크게 바꾸지 않고 조금씩 안정적으로 업데이트하는 알고리즘이다. 보정을 내는 actor 와 그 보정이 결과적으로 얼마나 좋았는지를 점수로

평가하는 critic 이 짝을 이뤄 critic 의 평가를 기준으로 actor 가 더 나은 보정을 내도록 학습된다.

5. 학습 신호인 reward 는 스킬별로 핵심 단계를 설정해 각 단계에 도달시 milestone 보상을 크게 주고 목표까지의 거리, 자세가 좋아지는 방향으로 작은 step 당 shaping 점수를 더해 설계한다. 최종적으로 정책은 이 보상을 최대화하도록 학습된다.

핵심 수치:

```
구조 : DP based frozen BC + ResidualPolicy(MLP 2×256, obs + base action
input)
보정값 action = base_action + clamp(-1~1 의 보정된 정규화 값) × per_dim_scale 이
후 역정규화로 action 값 생성
per_dim_scale (차원별 보정 한도):
    navigate : 팔 0.05 / 그리퍼 0.05 / 바퀴 0.0 (base 는 BC 그대로)
    approach&lift : 팔 0.20 / 그리퍼 0.25 / 바퀴 0.35
    carry : 팔 0.05 / 그리퍼 0.05 / 바퀴 0.0
    approach&place : 4 단계 ABCD - A 접근 바퀴 0.40 / B 하강 팔 0.05·바퀴 0.20 / C
    놓기 팔 0.05·그리퍼 0.50·바퀴 0.05 / D 후퇴 팔 0.10·그리퍼 0.30·바퀴 0.05
RL 학습(PP0) : train_resip.py · clip 0.2 · γ 0.999 · lr 보정 3e-4 / critic
5e-3 · KL 0.1 서 조기종단
```

6. Navigate

Navigate skill 은 전진/후진/왼쪽/오른쪽/시계방향/반시계방향 이동 총 6 가지 행동으로 구성되어있으며 최초에 각 행동당 20 개의 데모를 진행해 수집 후 이를 BC -> RL 을 진행해 expert 를 생성한다.

Navigate 시 팔은 카메라를 가리지 않을 정도로 들어올린 후 그 상태를 거의 유지하며 진행하고 단순한 행동으로 BC 에서도 높은 성공률을 보인다.

명령어

데모수집

```
python collect_nav_skill2.py --output
demos/navigate_skill2_120ep_26d.hdf5
```

bc 학습

```
python train_diffusion_bc.py \  
  --demo_path demos/navigate_skill2_120ep_26d.hdf5 \  
  --obs_dim 26 \  
  --down_dims 64 128 256 \  
  --save_dir checkpoints/dp_bc_nav_skill2_v4 \  
  --epochs 300
```

bc 검증

```
python eval_navigate.py \  
  --skill navigate \  
  --dp_checkpoint checkpoints/dp_bc_nav_skill2_v4/dp_bc_epoch300.pt \  
  --num_envs 1 \  
  --num_episodes 6 \  
  --inference_steps 8 \  
2> >(grep -v "omni.physx.tensors.plugin")
```

rl

```
PYTHONUNBUFFERED=1 nohup python train_resip.py \  
  --skill navigate \  
  --bc_checkpoint checkpoints/dp_bc_nav_skill2_v4/dp_bc_epoch300.pt \  
  --num_envs 512 \  
  --num_env_steps 200 \  
  --save_dir checkpoints/resip_nav_tucked_v4 \  
  --total_timesteps 50000000 \  
  --ent_coef 0.001 \  
  --target_kl 0.02 \  
  --normalize_reward True \  
  --headless \  
> /tmp/nav_rl_train.log 2>&1 &
```

rl 검증

```
python eval_navigate.py \  
  --skill navigate \  
  --dp_checkpoint checkpoints/dp_bc_nav_skill2_v4/dp_bc_epoch300.pt \  
  --resip_checkpoint checkpoints/resip_nav_tucked_v4/resip_nav_best.pt \  
\  
  --num_envs 1 \  
  --num_episodes 6 \  
  --inference_steps 8 \  
2> >(grep -v "omni.physx.tensors.plugin")
```

skill2env

[코드 — 노션 원문 참조 (1856 줄)]

skill2eval

[코드 — 노션 원문 참조 (2057 줄)]

collect nav_skill2

[코드 — 노션 원문 참조 (132 줄)]

traindiffusionbc

[코드 — 노션 원문 참조 (548 줄)]

evalnavigate

[코드 — 노션 원문 참조 (614 줄)]

trainresip

[코드 — 노션 원문 참조 (2465 줄)]

7. Approach&Lift

Approach&Lift 는 Navigate 로 약병과 그리 멀지 않은 지점까지 이동했을 때 물체를 들어올리기 위해서는 물체를 집어올릴 수 있는 지점까지 접근 후 팔을 내려 약병을 잡고 다시 들어올려 물체를 떨어트리지 않을 만한 상태의 tucked pose 를 취한 상태로 마무리 되어야 한다.

마찬가지로 최초에 각 행동당 20 개의 데모를 진행해 수집 후 이를 BC -> RL 을 진행해 expert 를 생성한다.

BC 진행시에 팔을 약병 근처까지 내려 잡는 행동을 취하고 이후 팔을 들어올려 tucked pose 를 취하는 흉내는 내지만 약병을 제대로 집어 성공하는 경우는 5% 정도밖에 되지 않아 이를 Residual RL 로 미세보정해 정확도를 끌어올린다.

record_teleop --skill combined 으로 한 에피소드를 돌면 코드가 phase 를 추적해 approach&lift 구간(phase1)을 combined_skill2_*.hdf5 로 자동 저장한다. lift 성공이 자동 판정되면 dest 를 앞에 자동 스폰하고 곧장 place 구간으로 넘어간다(carry 는 건너뛴). 별도 split 후처리는 필요 없다. --combined_manual_transit 옵션을 주면 carry transit(phase2)를 수동으로 거칠 수 있다.

명령어

데모수집

로컬 노트북에서 `python .\yourpath\Leader_to_home_with_wheel.py` 를 실행한 후 isaacsim 이 설치된 환경에서 다음 명령어를 실행한다. 아래 명령어는 한번에 `approach&lift`, `approach&place` 데모를 둘 다 수집하므로 한번에 수집 후

```
python record_teleop.py \
  --skill combined \
  --num_demos 20 \
  --teleop_source tcp \
  --listen_port 15002 \
  --object_scale_phys 0.7 \
  --object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
  --dest_object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_oz_cup/model_clean.usd \
  --gripper_contact_prim_path
"/World/envs/env_*/Robot/LeKiwi/Moving_Jaw_08d_v1" \
  --arm_limit_json calibration/arm_limits_measured.json
```

bc 학습

```
python train_diffusion_bc.py \
  --demo_path demos_skill2/combined_skill2_20260227_091123.hdf5 \
  --obs_dim 30 \
  --down_dims 64 128 256 \
  --epochs 300 \
  --save_dir checkpoints/dp_bc_small
```

bc 검증

```
python eval_resip.py \
  --skill approach_and_grasp \
  --dp_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
  --num_episodes 5 \
  --object_usd
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd
```

rl

rl 진행 시 물체를 집고 하늘로 쏘 뽐는 것 까지 잘하는 체크포인트 V1 학습 이후 이를 lifted pose 를 취하게 만드는 추가 학습 V2 를 resume 으로 진행한다. 또한 그때마다 skill2 env 와 train resip 코드가 다르므로 교체 후 진행해야 함

V1

```
cd /home/jovyan/IsaacLab/scripts/lekiwi_nav_env && \
PYTHONUNBUFFERED=1
LEKIWI_USD_PATH=/home/jovyan/Downloads/lekiwi_robot.usd \
nohup python train_resip.py \
--skill approach_and_grasp \
--bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
--object_usd
/home/jovyan/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
--num_envs 1024 \
--lr_actor 1e-3 \
--target_kl 1.5 \
--total_timesteps 250000000 \
--normalize_reward False \
--headless
```

V2

```
cd /home/jovyan/IsaacLab/scripts/lekiwi_nav_env && \
PYTHONUNBUFFERED=1
LEKIWI_USD_PATH=/home/jovyan/Downloads/lekiwi_robot.usd \
nohup python train_resip.py \
--skill approach_and_grasp \
--bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
--object_usd
/home/jovyan/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
--num_envs 512 \
--lr_actor 1e-4 \
--lr_critic 1e-3 \
--update_epochs 10 \
--num_minibatches 4 \
--target_kl 0.03 \
--ent_coef 1e-4 \
--total_timesteps 250000000 \
--normalize_reward True \
--resume_resip /home/jovyan/data/resip_v6.4b_best.pt \
--resume_actor_only True \
--warmup_steps_initial 0 \
--enable_domain_randomization False \
--r8_penalty -2.0 \
--save_dir checkpoints/resip_v6.4b_tuned_from66 \
--headless \
> logs/resip_v64b_tuned_from66_20260319_025055.log 2>&1 &
```

rl 검증

```
python eval_resip.py --skill approach_and_grasp \
--dp_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
--resip_checkpoint "backup/appoachandlift/resip64%.pt" \
--num_episodes 5 \
```

```
--object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_cl
ean.usd \
2> >(grep -v "omni.physx.tensors.plugin")
```

recordteleop

[코드 — 노션 원문 참조 (2124 줄)]

traindiffusionbc

[코드 — 노션 원문 참조 (548 줄)]

evalresip

[코드 — 노션 원문 참조 (710 줄)]

v1 resip_v6.4b_best.pt

[코드 — 노션 원문 참조 (1854 줄)]

lekiwiskill2

[코드 — 노션 원문 참조 (1854 줄)]

train

[코드 — 노션 원문 참조 (765 줄)]

eval

[코드 — 노션 원문 참조 (475 줄)]

v2 resip64%.pt

lekiwiskill2env

[코드 — 노션 원문 참조 (1961 줄)]

train

[코드 — 노션 원문 참조 (844 줄)]

eval

[코드 — 노션 원문 참조 (477 줄)]

8. Carry

Carry는 Approach&Lift에서 물체를 들고있는 tucked pose를 거의 유지하면서 Navigate와 마찬가지로 전진/후진/왼쪽/오른쪽/시계방향/반시계방향 이동 총 6가지 행동을 물체를 떨어트리지 않는 상태를 유지하며 진행하는 스킬이다.

마찬가지로 최초에 각 행동당 20개의 데모를 진행해 수집 후 이를 BC -> RL을 진행해 expert를 생성한다.

검증과 rl 학습은 Approach&Lift 진행 이후 tucked pose를 취한 상태에서부터 진행되는데 이때 문제는 시작과 동시에 컵을 떨어트리거나 arm과 gripper가 고장나는 경우이다. 이 원인은 Approach&Lift에서 물체를 집고 있는 상태에서 넘어올 때 그 관절값이 그때그때마다 다르게 되는데 수집한 20개의 데모 분포에서 조금 벗어나면 모방학습이 본 적 없는 분포이기 때문에 고장나는 것이다. 이를 해결하기 위해 데이터 수집시에 관측 OBS에 시작 관절값들을 넣어 그 시작 관절값을 유지하는 것을 학습할 수 있도록 진행했고 또 수집한 20개의 분포에서 벗어나더라도 어느정도 강건하게 진행될 수 있게 20개의 데모의 값에 노이즈를 미세하게 노이즈를 넣어 oversampling해 데이터를 증폭시켰다. 그 결과 BC 및 RL 진행 이후 물체를 떨어트리지 않고 꽤 높은 성공률을 보이게 되었다.

명령어

carry data record

명령어

```
PYTHONUNBUFFERED=1 nohup python collect_carry.py \
--s2_bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
```

```

--s2_resip_checkpoint backup/appoachandlift/resip64%.pt \
--object_usd
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
--num_envs 1 --num_reps 20 \
--carry_steps 200 \
--output demos/carry_120ep_39d.hdf5 \
--headless \
> /tmp/carry_collect.log 2>&1 &
echo "PID: $!"

```

skill2eval

[코드 — 노션 원문 참조 (2057 줄)]

collect carry

[코드 — 노션 원문 참조 (362 줄)]

carry BC

명령어

학습

```

python train_diffusion_bc.py \
--demo_path demos/carry_120ep_39d.hdf5 \
--obs_dim 39 \
--down_dims 64 128 256 \
--save_dir checkpoints/dp_bc_carry_v4 \
--epochs 300 \
2>&1 | grep -E "Final|Error"

```

검증

```

python eval_carry.py \
--carry_bc_checkpoint checkpoints/dp_bc_carry_v4/dp_bc_epoch300.pt \
--s2_bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
--s2_resip_checkpoint backup/appoachandlift/resip64%.pt \
--object_usd
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
--num_episodes 6 \
2> >(grep -v "omni.physx.tensors.plugin")

```

traindiffusionbc

[코드 — 노션 원문 참조 (548 줄)]

evalcarry

[코드 — 노션 원문 참조 (296 줄)]

carry PPO

명령어

train

```
PYTHONUNBUFFERED=1 nohup python train_resip.py \  
    --skill carry \  
    --bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \  
    --s2_resip_checkpoint backup/appoachandlift/resip64%.pt \  
    --s3_bc_checkpoint \  
checkpoints/dp_bc_carry_v4/dp_bc_epoch300.pt \  
    --object_usd \  
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \  
    --gripper_contact_prim_path \  
"/World/envs/env_*/Robot/LeKiwi/Moving_Jaw_08d_v1" \  
    --num_envs 512 \  
    --num_env_steps 1500 \  
    --save_dir checkpoints/resip_carry_v6 \  
    --total_timesteps 50000000 \  
    --ent_coef 0.001 \  
    --target_kl 0.02 \  
    --normalize_reward True \  
    --headless \  
> /tmp/carry_rl_train.log 2>&1 &  
echo "PID: $!"
```

eval

```
python eval_carry.py \  
    --carry_bc_checkpoint checkpoints/dp_bc_carry_v4/dp_bc_epoch300.pt \  
    --carry_resip_checkpoint \  
checkpoints/resip_carry_v6/resip_carry_iter240.pt \  
    --s2_bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \  
    --s2_resip_checkpoint backup/appoachandlift/resip64%.pt \  
    --object_usd \  
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \  
    --num_episodes 6 \  

```

```
2> >(grep -v "omni.physx.tensors.plugin")
```

skill2eval

[코드 — 노션 원문 참조 (2057 줄)]

train

[코드 — 노션 원문 참조 (2479 줄)]

evalcarry

[코드 — 노션 원문 참조 (296 줄)]

9. Approach&Place

Approach&Place 는 Carry 로 dest 물체 근처까지 옮겨온 물체를, dest 옆에 정밀하게 다가가 그리퍼를 열어 내려놓는 마지막 스킬이다. 단순히 이동만 하는 것이 아니라, dest 에 도착한 뒤 접근 -> 하강 -> 놓기 -> 후퇴의 4 단계(ABCD)로 물체를 정해진 위치에 안착시킨다.

마찬가지로 데모를 수집해 BC -> RL 로 expert 를 생성한다.

검증과 rl 학습은 Carry 성공 상태(handoff, 물체를 든 채)에서 시작하므로, Carry 와 똑같이 시작 관절값이 데모 분포를 조금만 벗어나도 컵을 떨어트리거나 arm 과 gripper 가 고장나는 문제가 있다. 그래서 OBS 에 시작 관절값들을 넣어 그 자세를 유지하도록 학습한다. 다만 데이터 증폭 방식은 Carry 와 다르다. Carry 는 collect_carry.py 가 수집 시점에 arm/base action 에 노이즈 (arm 0.02 / base 0.005)를 더해 증폭하지만, Place 데모는 노이즈가 아니라 make_skill3_*.py 로 특정 구간(release/retract/하강 등)을 복제·반복(duplication/boost)해 증폭하고, BC 학습 때 phase_a_oversample 로 Phase A 샘플을 추가 복제한다(노이즈 주입 아님).

다만 Approach&Place 는 정밀하게 내려놓는 것이 핵심이라 관측을 55D 로 늘려, 시작 자세와 source/dest 상대위치, 지금 어느 단계인지를 알리는 플래그까지 넣어 단계마다 무엇을 해야 하는지를 보게 했다. 보상도 dest 옆 정해진 거리·높이에 물체를 똑바로 세워 놓고 팔을 접었을 때 크게 주도록 설계했다(환경·관측·보상의 자세한 구조는 §4.6).

같은 combined 텔레옴에서 place 구간(phase3)이 자동으로 combined_skill3_*.hdf5에 따로 저장된다. 즉 approach&lift와 place 데모가 한 번의 텔레옴으로 동시에 분리 수집된다. carry 구간(phase2/transit)은 미기록이며 carry 데모는 collect_carry.py로 별도 수집한다. --combined_manual_transit 옵션을 주면 carry transit를 수동으로 거칠 수 있다.

[VLA 데이터 미수집] approach&place(S4)의 VLA 파인튜닝 데이터는 아직 수집하지 않았다. scene_expert_combined_s2_place 텔레옴 데이터는 있으나 VLA 학습셋 lekiwi_viva_v5에는 미포함이며(13 task = navigate 6 + carry 6 + approach 1, place=0), place는 향후 데이터 수집 후 진행한다.

명령어

approachandplace record teleop

명령어

full

이 데모 수집은 앞선 approach&lift 수집시에 이미 다 진행되었을 것이다.

```
python record_teleop.py \
  --skill combined \
  --num_demos 20 \
  --s3_obs_mode phased53 \
  --object_scale_phys 0.7 \
  --dest_object_scale 0.56 \
  --gripper_contact_prim_path
"/World/envs/env_*/Robot/LeKiwi/Moving_Jaw_08d_v1" \
  --object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
  --dest_object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_oz_cup/model_clean.usd
```

resume

```
python record_teleop.py \
  --skill combined \
  --resume \
  --output
/home/yubin11/IsaacLab/scripts/lekiwi_nav_env/demos/combined_skill3_20260408_101525.hdf5 \
  --num_demos 20 \
  --s3_obs_mode phased53 \
```

```
--gripper_contact_prim_path
"/World/envs/env_*/Robot/LeKiwi/Moving_Jaw_08d_v1" \
--object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
--dest_object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_oz_cup/model_clean.usd
```

record_teleop.py

[record_teleop.py — 노션 원문 참조]

skill2env

[코드 — 노션 원문 참조 (1856 줄)]

skill2eval

[코드 — 노션 원문 참조 (2061 줄)]

skill3_bc_obs

[코드 — 노션 원문 참조 (517 줄)]

approachandplace BC

명령어

```
python train_diffusion_bc.py \
--demo_path demos/combined_skill3_55d_final.hdf5 \
--obs_dim 55 \
--act_dim 9 \
--pred_horizon 16 \
--action_horizon 8 \
--epochs 500 \
--batch_size 1024 \
--lr 1e-4 \
--phase_a_oversample 1 \
--phase_a_flag_idx 35 \
--save_dir checkpoints/dp_bc_skill3_55d_fixed_1e-4 \
--save_name dp_bc.pt \
--save_every 50
```

eval

```
PYTHONUNBUFFERED=1 python eval_s3.py \
  --s3_bc_checkpoint
checkpoints/dp_bc_skill3_55d_fixed_1e-4/dp_bc_epoch500.pt \
  --s2_bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
  --s2_resip_checkpoint backup/approachandlift/resip64%.pt \
  --object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
  --object_scale_phys 0.7 \
  --dest_object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_oz_cup/model_clean.usd \
  --dest_object_scale 0.56 \
  --num_envs 500 \
  --num_episodes 500 \
  --headless \
  --results_jsonl ~/eval_s3_bc_55d_v6_pa20_300_200.jsonl \
  --trace_jsonl ~/eval_s3_bc_55d_v6__pa20_300_200.trace.jsonl \
  --s3_motion_release_xy 0.12 \
  --s3_motion_release_ee_z 0.10 \
  --s3_motion_retract_grip 0.90 \
  2>&1 | grep --line-buffered -v "omni.physx.tensors.plugin" | tee
~/eval_s3_bc_55d_v6_pa20_300_200.log
```

train

[코드 — 노션 원문 참조 (548 줄)]

diffusion_policy.py

[diffusion_policy.py — 노션 원문 참조]

eval_s3

[코드 — 노션 원문 참조 (1651 줄)]

approachandplace PPO

명령어

ver v25

```

PYTHONUNBUFFERED=1 python train_resip.py \
--skill combined_s2_s3 \
--bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
--s2_resip_checkpoint "backup/appoachandlift/resip64%.pt" \
--s3_bc_checkpoint
checkpoints/dp_bc_skill3_55d_fixed_1e-4/dp_bc_epoch500.pt \
--object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
--dest_object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_oz_cup/model_clean.usd \
--num_envs 1024 --num_env_steps 3000 --total_timesteps 200000000 \
--s2_lift_hold_steps 200 --s3_curriculum_stage v15_dense \
--normalize_reward True --init_logstd -2.0 \
--logstd_anneal_to -3.2 --logstd_anneal_start 0.3 \
--lr_actor 1e-3 --lr_critic 5e-3 --target_kl 1.5 --ent_coef 0.001 \
--save_dir checkpoints/resip_s3_v25 --seed 82 --headless

```

rl eval

```

PYTHONUNBUFFERED=1 python eval_s3.py \
--s3_bc_checkpoint
checkpoints/dp_bc_skill3_55d_fixed_1e-4/dp_bc_epoch500.pt \
--s3_resip_checkpoint checkpoints/resip_s3_v25/resip_final.pt \
--s2_bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
--s2_resip_checkpoint "backup/appoachandlift/resip64%.pt" \
--object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
--object_scale_phys 0.7 \
--dest_object_usd
/home/yubin11/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_oz_cup/model_clean.usd \
--dest_object_scale 0.56 \
--inference_steps 4 \
--num_envs 1 --num_episodes 20 \
2>&1 | grep --line-buffered -v "omni.physx.tensors.plugin" | tee
~/eval_s3_v25_watch.log

```

skill2env

[코드 — 노션 원문 참조 (1856 줄)]

skill2eval

[코드 — 노션 원문 참조 (2061 줄)]

skill3_bc_obs

[코드 — 노션 원문 참조 (517 줄)]

train_resip v19t

[코드 — 노션 원문 참조 (3778 줄)]

train_resip v19t 50 + v19u 50 iter 100

[코드 — 노션 원문 참조 (3778 줄)]

train_resip v25 iter 80

[코드 — 노션 원문 참조 (3879 줄)]

eval_s3

[코드 — 노션 원문 참조 (1651 줄)]

eval_s3 for v25

[코드 — 노션 원문 참조 (1670 줄)]

10. Record Teleop Scene

구축한 RL Expert 를 ProcTHOR scene(molmospaces, scene_idx 1302)에서 자동 rollout 시켜 모방학습용 HDF5 데이터를 수집한다. --only_success 는 성공 에피소드만 저장한다.

--skill 선택지: navigate / approach_and_grasp / carry_and_place / combined_s2_s3 / combined_s2_place / full. difficulty-aware spawn(--difficulty: easy / middle / hard, easy·middle=같은 방, hard=다른 방)과 full end-to-end 텔레옴 (--skill full, _FULL_ROOM_SCHEDULE)을 추가로 지원한다. 10.1 navigate 명령의 플래그(--bc_checkpoint / --resip_checkpoint / --scene_idx / --scene_scale / --

num_demos / --max_episode_steps)는 모두 argparse 에 존재한다. 10.4 의 --skill combined_s2_place 도 유효한 choices 로, approach & place 데이터 수집에 사용한다.

10.1 Navigate (S1)

```
PYTHONUNBUFFERED=1 python vllm/record_teleop_scene.py \  
  --skill navigate \  
  --bc_checkpoint checkpoints/dp_bc_nav_skill2_v4/dp_bc_epoch300.pt \  
  --resip_checkpoint checkpoints/resip_nav_tucked_v4/resip_nav_best.pt \  
 \  
  --scene_idx 1302 \  
  --scene_scale 1.0 \  
  --num_demos 480 \  
  --max_episode_steps 150 \  
2> >(grep -v "omni.physx.tensors.plugin")
```

10.2 Approach & Lift (S2)

```
PYTHONUNBUFFERED=1 python vllm/record_teleop_scene.py \  
  --skill approach_and_grasp \  
  --bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \  
  --resip_checkpoint backup/appoachandlift/resip64%.pt \  
  --object_usd \  
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \  
  --scene_idx 1302 \  
  --scene_scale 1.0 \  
  --num_demos 100 \  
  --only_success \  
2> >(grep -v "omni.physx.tensors.plugin")
```

10.3 Carry (combined_s2_s3)

```
PYTHONUNBUFFERED=1 python vllm/record_teleop_scene.py \  
  --skill combined_s2_s3 \  
  --bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \  
  --resip_checkpoint backup/appoachandlift/resip64%.pt \  
  --bc_checkpoint_s3 checkpoints/dp_bc_carry_v4/dp_bc_epoch300.pt \  
  --resip_checkpoint_s3 checkpoints/resip_carry_v6/resip_carry_iter240.pt \  
  --object_usd \  
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \  
  --scene_idx 1302 \  
  --scene_scale 1.0 \  
  --num_demos 100 \  
  --only_success \  
2> >(grep -v "omni.physx.tensors.plugin")
```

10.4 Approach & Place (combined_s2_place)

```
PYTHONUNBUFFERED=1 python vllm/record_teleop_scene.py \
  --skill combined_s2_place \
  --bc_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
  --resip_checkpoint "backup/appoachandlift/resip64%.pt" \
  --bc_checkpoint_s3
checkpoints/dp_bc_skill3_55d_fixed_1e-4/dp_bc_epoch500.pt \
  --resip_checkpoint_s3 checkpoints/resip_s3_v25/resip_iter80.pt \
  --object_usd
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
  --dest_object_usd
~/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_
oz_cup/model_clean.usd \
  --scene_idx 1302 \
  --num_demos 200 \
  --only_success \
  2> >(grep -v "omni.physx.tensors.plugin")
```

vllm/record_teleop_scene.py

[scene.py — 노션 원문 참조]

11. 데이터 후처리

수집한 스킬별 HDF5 를 하나로 합치고 LeRobot v3 데이터셋으로 변환한 뒤 정규화 통계를 보정한다.

11.1 HDF5 Merge

```
# [사용법] 아래를 merge.py 로 저장 후 lerobot050 env에서 실행: conda activate
lerobot050 && python merge.py
# inputs 리스트에 $10 에서 수집한 스킬별 HDF5 파일 경로들을 넣는다. output 은 합쳐진 파일
명.
# 스킬별 HDF5 를 하나로 합친다 (에피소드 인덱스 재부여)
import h5py
output = "viva_merged.hdf5"
inputs = ["100success_approach_lift.hdf5", "navigate_scene_480ep.hdf5",
"carry_scene.hdf5"]
fo = h5py.File(output, "w"); ep_idx = 0
for fname in inputs:
    fi = h5py.File(fname, "r")
    for key in sorted(fi.keys(), key=lambda x: int(x.split("_")[1])):
        fi.copy(key, fo, name=f"episode_{ep_idx}"); ep_idx += 1
    fi.close()
fo.attrs["total_episodes"] = ep_idx; fo.close()
```

11.2 LeRobot v3 변환

```
# [사용법] lerobot050 env에서 실행. --input 에 11.1 merge 결과(또는 스킵별 *.hdf5
그룹)를 지정.
conda activate lerobot050
# HDF5 → LeRobot v3 (Parquet+MP4). 단위 변환 불필요(sim m/s = real m/s),
gripper continuous
python convert_hdf5_to_lerobot_v3.py \
  --input outputs/rl_demos/*.hdf5 \
  --output_root outputs/lerobot_v3/lekiwi_skill2 \
  --repo_id local/lekiwi_skill2 \
  --fps 25 \
  --include_subtask_index \
  --skip_episodes_without_images
```

convert_hdf5_to_lerobot_v3.py

[v3.py — 노션 원문 참조]

11.3 정규화 (stats.json q01/q99 보정)

수집 데이터를 LeRobot 데이터 형식으로 변환한 뒤 플랫폼 형식에 맞춰 자동 계산된 stats.json 은 q01/q99 가 팔은 거의 움직이지 않고 바퀴만 움직이는 navigate, carry 데이터에 쏠려 있어 approach&lift, approach&place 의 arm 상태가 정규화시 폭주한다 (예: arm4 실제값 +0.534 → 정규화 +5.60).

이러한 문제를 해결하기 위해 학습 전 q01/q99 를 전체 데이터 범위 + 5% 마진으로 확장한다.

```
# stats.json q01/q99 → 전체 min/max + 5% 마진으로 확장 (lerobot050 env에서 실행)
# [사용법] 아래를 fix_stats.py 로 저장 → conda activate lerobot050 && python
fix_stats.py → 이어서 아래 검증 스크립트 실행
import json, shutil, numpy as np
import pyarrow.parquet as pq

DATA_ROOT = 'outputs/lerobot_v3/lekiwi_skill2' # 변환 출력 경로

stats = json.load(open(f'{DATA_ROOT}/meta/stats.json'))
data = pq.read_table(f'{DATA_ROOT}/data/chunk-000/file-000.parquet')
states = np.array(data.column('observation.state').to_pylist())
actions = np.array(data.column('action').to_pylist())

for feat_key, feat_data in [('observation.state', states), ('action',
actions)]:
```

```

        for i in range(9):
            # 9D: arm0-4, gripper,
            base_vx, base_vy, base_wz
            lo = float(feats_data[:, i].min()); hi = float(feats_data[:,
            i].max())
            margin = (hi - lo) * 0.05
            stats[feat_key]['q01'][i] = lo - margin
            stats[feat_key]['q99'][i] = hi + margin

shutil.copy(f'{DATA_ROOT}/meta/stats.json',
f'{DATA_ROOT}/meta/stats_backup.json')
with open(f'{DATA_ROOT}/meta/stats.json', 'w') as f:
    json.dump(stats, f, indent=2)
print("stats.json 갱신 완료. 원본 백업: stats_backup.json")

# [사용법] verify_stats.py 로 저장 후 conda activate lerobot050 && python
verify_stats.py 로 정규화 범위 확인
# 정규화 범위 확인 — 모든 차원이 대략 [-0.3, 1.3] 안에 들어와야 정상 (보정 전 arm4
max=+5.60)
import json, numpy as np
import pyarrow.parquet as pq

DATA_ROOT = 'outputs/lerobot_v3/lekiwi_skill2'
stats = json.load(open(f'{DATA_ROOT}/meta/stats.json'))
data = pq.read_table(f'{DATA_ROOT}/data/chunk-000/file-000.parquet')
states = np.array(data.column('observation.state').to_pylist())
actions = np.array(data.column('action').to_pylist())

for feat_key, feat_data in [('observation.state', states), ('action',
actions)]:
    q01 = np.array(stats[feat_key]['q01']); q99 =
np.array(stats[feat_key]['q99'])
    normalized = (feat_data - q01) / (q99 - q01 + 1e-8)
    print(f"\n{feat_key}:")
    for i in range(9):
        col = normalized[:, i]
        print(f"  dim{i}: min={col.min():.3f}  max={col.max():.3f}")

```

11.4 VLA 데이터셋 BUILD PIPELINE (lekiwi_viva_v5)

VLA(Pi0.5) 파인튜닝 데이터셋 lekiwi_viva_v5 는 위 11.1~11.3(로컬 convert)와 별개로, 서버 전용 파이프라인 vllm/dataset_tools/ 로 빌드한다. 로컬 레포에는 없고 서버에만 있다. 4 개 스크립트로 hdf5_to_lerobot → build_v5 → remap_v5_tasks → verify_v5 순서로 진행한다.

1. record_teleop_scene.py (§ 10): BC+ResiP expert rollout 으로 scene_expert_*.hdf5 를 수집한다. 각 에피소드에 actions(T,9), robot_state(T,9), images/base_rgb·wrist_rgb(T,400,640,3 uint8), attrs\[instruction\] 가 들어간다.

2. hdf5_to_lerobot.py: scene_expert_*.hdf5 → LeRobot v3.0. observation.state 9D + action 9D + observation.images.front/wrist(400×640, video, pyav). lekiwi_viva_v4 스키마와 동일(robot_type=lekiwi_client, fps=25). episode 단위 add_frame → save_episode → finalize.

사용: python hdf5_to_lerobot.py --hdf5 \<scene_expert.hdf5> --out \<ds_root>

3. build_v5.py: lekiwi_viva_v5 = filtered v4(old approach 0..99 drop) + new approach(local 50 + server 50) 병합. LeRobotDataset.create + add_frame + save_episode 으로 lerobot 자체가 chunking/video/stats 처리. v4 task 순서를 유지해 task_index 정합 보존. finalize 후 정확 quantile stats(q01/q10/q50/q90/q99) 재계산해 stats.json 에 baking.

사용: python build_v5.py --v4 \<v4_root> --new-sources \<ds1> \<ds2> --out \<v5_root> --drop-v4-episodes 0-99

4. remap_v5_tasks.py: tasks.parquet 을 arm-state 명시 instruction 으로 재작성. navigate forward → navigate forward with arm tucked, carry forward → carry forward holding the object 등(nav=+ with arm tucked, carry=+ holding the object, approach&lift=그대로). 이 remap 이 VLA 라벨을 VLM orchestrator 출력과 exact-match 시키는 핵심이다.

사용: python remap_v5_tasks.py --path \<v5_root> --backup

5. verify_v5.py: 무결성 검사(info/episodes/stats 일관성, state/action NaN·inf 없음, global index contiguous, frame_index 0 부터 연속, task 문자열 ↔ tasks.parquet 매칭, stats q01/q99/mean/std tolerance, video decode, LeRobotDataset getitem 샘플). 실패 시 non-zero exit.

사용: python verify_v5.py --path \<v5_root>

결과: viva_v5 = 978 ep (navigate 446 + carry 432 + approach 100), 13 task
(navigate 6 + carry 6 + approach&lift 1). place(S4) task 는 0 — VLA 데이터 미수집
(§ 9 참조). place 는 향후 데이터 수집 후 추가 빌드한다.